빌드, 구성, 운영과 복원 저장소 읽기

번들에서 복원된 코드의 한계를 인정하면서 feature gate·설정·migration·telemetry·리팩터링 신호를 실제 프로젝트 운영 구조로 읽는다

이 권을 마치면 보이는 코드를 공식 원본이나 활성 제품과 동일시하지 않고도, build-time variability와 운영 계측이 설계를 어떻게 형성하는지 구체적으로 설명할 수 있다.

STUDY SNAPSHOT
Claude Code v2.1.88
2ca5ddabfed5

제3자 복원 소스 기반
한국어 · Guided Edition

오늘 우리는 이 코드의 어느 부분을 함께 보는가

이 저장소는 정상적인 개발 checkout이 아니라 배포 npm 번들에서 복원된 v2.1.88 snapshot이다. scripts와 stubs는 그 차이를 메우려 하지만 누락된 내부 모듈과 build intrinsic을 완전히 복구할 수 없다. 이 제약을 분석의 변명으로 쓰지 않고, 무엇이 관찰이고 무엇이 추론인지 분리하는 출발점으로 삼는다.

267	43,983	288	22
분류된 TS/TSX 파일	물리 코드 줄	async 함수	tool use 표기

CHAPTER 01

복원 저장소의 출처와 한계를 먼저 고정한다

코드가 존재한다는 사실, 원래 source tree와 같다는 주장, production에서 활성이라는 주장은 서로 다른 증거를 요구한다.

CHAPTER 02

구성과 migration이 제품 변형을 운영한다

feature flag, settings, model alias, command registration은 코드 밖 결정이 runtime 행동을 바꾸게 하는 구성 언어다.

CHAPTER 03

Telemetry와 비용 코드는 운영의 두 번째 모델이다

실행 코드가 무엇을 하는지와 운영 코드가 무엇을 관찰하는지는 다르며, 관찰 schema가 팀의 문제 인식을 형성한다.

CHAPTER 04

연구 문서에서 구조 개선 과제로 돌아온다

발견 목록을 그대로 믿거나 버리지 않고, 그 문서의 탐색법을 core code와 연결해 검증 가능한 리팩터링 질문으로 바꾼다.

분포 수치는 중요도 점수가 아니다. 이 권이 직접 펼쳐보는 코드 표면의 규모와 언어 메커니즘 밀도를 가늠하기 위한 탐색 표지다.

01

복원 저장소의 출처와 한계를 먼저 고정한다

코드가 존재한다는 사실, 원래 source tree와 같다는 주장, production에서 활성이라는 주장은 서로 다른 증거를 요구한다.

이 장에서 실제로 펼쳐볼 경로

README_KR.md · QUICKSTART.md · package.json · scripts/** · stubs/**

이 snapshot은 무엇이고 무엇이 아닌가

npm 패키지 2.1.88의 단일 bundle에서 TypeScript source 형태를 복원한 제3자 저장소다.

README는 버전과 추출 경로, 약 1,884개 TS/TSX 파일, 누락된 feature-gated 모듈을 명시한다. 우리가 직접 고정한 commit은 해당 저장소의 특정 상태일 뿐 Anthropic 내부 monorepo revision은 아니다.

따라서 함수와 call site는 이 snapshot에서의 관찰 사실이다. 원래 파일 경계, 주석의 원작성, production cohort, 미래 roadmap은 더 약한 추론이다. 보고서 전반에서 이 차이를 유지한다.

연구 자료로서는 충분히 가치가 있다. 단, “Claude Code는 반드시 이렇게 동작한다”보다 “이 배포 snapshot의 복원 코드는 이렇게 구성되어 있다”고 말하는 것이 정확하다.

스터디에서 확인할 질문

한 코드 경로가 존재한다는 사실로 실제 사용자에게 활성화되었다고 결론 내리려면 어떤 추가 evidence가 필요한가?

feature()는 runtime if가 아니라 build-time variability다

Bun intrinsic이 false로 결정되면 branch와 import가 dead-code elimination 대상이 된다.

README와 QUICKSTART는 내부 build에서 `feature(FLAG)`가 compile time에 해석된다고 설명한다. 배포 bundle에는 false branch의 module이 아예 없을 수 있으므로 source reference만 남고 구현은 복원할 수 없다.

이 메커니즘은 software product line의 variation point와 닮았다. 하나의 source가 내부/외부, 실험 cohort, platform에 따라 다른 product를 만든다. runtime flag와 달리 배포 후 켤 수 없는 path도 있다.

분석에서는 missing import를 구현 부재로 기록해야지 임의 stub의 행동으로 추정하면 안 된다. stubs는 build 탐색용이지 원래 semantics의 evidence가 아니다.

Build-time gate가 module graph를 바꾸는 방식

분석용 TypeScript — 실제 구현의 제어 구조를 축약·재작성

```
01  if (feature("INTERNAL_MODE")) {
02    const module = require("./internal/worker")
03    start(module)
04  }
05
06  // external build
07  feature("INTERNAL_MODE")  →  false
08  false branch               →  removed
```

1. constant folding
   bundler가 feature 호출을 상수로 치환한다.
2. DCE
   도달 불가능한 branch와 module reference가 bundle에서 사라진다.
3. reconstruction
   남은 import 이름은 구현 내용을 복구하는 evidence가 아니다.

언어·런타임 안쪽에서는 compile-time intrinsic은 프로그램 실행 전에 module graph를 바꾼다. runtime flag와 달리 배포 후 값을 바꿔도 제거된 code는 돌아오지 않는다.

스터디에서 확인할 질문

같은 feature 이름이 build-time intrinsic과 runtime GrowthBook gate에 모두 쓰일 때 두 층을 어떻게 구분할 것인가?

prepare와 build script는 서로 다른 복원 전략을 보인다

원본을 직접 patch하는 경로와 복사본을 transform하는 경로의 안전성이 다르다.

`prepare-src.mjs`는 src 안의 bun import와 MACRO를 바꾸는 반면 `build.mjs`는 build-src로 복사해 transform하고 iterative stub을 만든다. 이름이 비슷해도 source mutation semantics가 다르다.

복원 도구는 대상 코드를 이해하기 위한 보조물이어야 한다. 원본 snapshot을 바꾸면 이후 분석의 provenance가 흔들린다. 그래서 이번 연구에서는 target build를 실행하지 않고 fixed clone을 read-only evidence로 사용했다.

범용 분석 도구도 repo-specific 가정을 코드에 하드코딩하지 않고 declarative grouping을 입력으로 받아야 다른 저장소에서 재사용할 수 있다. 분석기와 연구 질문을 분리하는 원칙이다.

두 스크립트의 source mutation 차이

경로	대상	효과	분석 위험
prepare-src	src/**	Bun import/MACRO 직접 치환	원본 snapshot 변형
build	build-src/**	복사본 transform	비교적 provenance 보존
stub loop	missing paths	빈 module 생성	semantic equivalence 아님

구분해야 하는 이유 빌드 성공은 syntax/module closure의 증거일 수 있지만 누락 구현의 behavior를 증명하지 않는다.

스터디에서 확인할 질문

prepare-src가 원본을 수정하는 현재 script name과 package script 조합은 사용자에게 어떤 오해를 줄 수 있는가?

함께 펼쳐볼 자료 scripts/prepare-src.mjs · scripts/build.mjs · package.json

stub이 build를 통과시켜도 의미는 복원되지 않는다

missing module에 빈 export를 만들면 linker 오류는 줄지만 호출 결과와 side effect는 발명된다.

build script는 esbuild 오류에서 missing path를 수집해 stub을 만든다. 이는 남은 source를 탐색 가능한 bundle로 만드는 best-effort 전략이지, 기능적으로 동일한 CLI를 증명하는 과정이 아니다.

특히 security, coordinator, daemon처럼 누락된 모듈은 empty function으로 대체했을 때 fail-open이나 silent no-op가 될 수 있다. 실행 성공을 기능 성공으로 읽으면 가장 위험하다.

재구성 build를 검증하려면 upstream binary와 differential behavior를 비교해야 하지만 인증·외부 효과·버전 제약이 있다. 비교할 수 없는 영역은 unknown으로 남겨야 한다.

스터디에서 확인할 질문

stubbed function이 호출되지 않았음을 증명하지 못한 build를 어떤 용도로까지 허용할 수 있는가?

02

구성과 migration이 제품 변형을 운영한다

feature flag, settings, model alias, command registration은 코드 밖 결정이 runtime 행동을 바꾸게 하는 구성 언어다.

이 장에서 실제로 펼쳐볼 경로

src/utils/settings/** · src/migrations/** · src/commands/** · src/utils/model/**

feature flag 이름보다 evaluation site를 읽는다

난독화된 flag도 call site와 fallback, audience를 추적하면 실제 영향을 알 수 있다.

저장소 문서는 `tengu_*` flag 목록을 발견 중심으로 정리한다. 스터디에서는 이름에서 의미를 추정하기보다 getter가 어디서 호출되고 false/unknown에서 무엇을 하는지 따라간다.

flag는 코드 배포 없이 behavior를 바꾸지만 테스트 상태 공간을 곱한다. 두 개의 독립 gate와 환경 변수 조합만으로도 여러 configuration이 생긴다.

중요 기능에는 configuration matrix와 invariant test가 필요하다. 모든 조합을 exhaustive하게 돌리지 못하면 위험 기반 pairwise와 default/failure path를 우선한다.

Flag의 실제 의미를 찾는 추적 순서

01	→	02	→	03	→	04	→	05
Definition default와 source		**Evaluation** build/start/turn/call 중 시점		**Call site** 켜질 때 생기는 capability		**Fallback** unknown/off/stale의 행동		**Observation** 어떤 event로 확인 가능한가

이 그림이 감추지 않는 것 flag 이름은 의미를 소유하지 않는다. evaluation time과 fallback이 실제 availability·safety semantics를 결정한다.

스터디에서 확인할 질문

voice mode처럼 build gate와 runtime killswitch가 겹친 기능의 실제 활성 조건을 boolean expression으로 적어 보라.

함께 펼쳐볼 자료 docs/ko/02-숨겨진-기능과-코드네임.md · src/services/analytics/growthbook.ts · src/voice/voiceModeEnabled.ts

설정 계층은 값 병합이 아니라 authority 병합이다

default, user, project, local, managed source는 같은 key라도 수정 권한이 다르다.

Settings 유틸리티는 여러 위치의 설정을 읽고 병합한다. 단순 object spread는 편하지만 어떤 source가 값을 이겼는지 잃을 수 있다.

특히 permission과 telemetry, model 선택은 provenance가 필요하다. 사용자가 UI에서 값을 바꿀 수 있어 보여도 managed layer가 덮으면 변경은 반영되지 않는다. UI가 effective value와 source를 함께 보여 줘야 한다.

설정 구조를 리팩터링할 때 parsed value, source metadata, validation error를 분리하면 fallback과 설명이 쉬워진다. invalid high-priority value를 낮은 값으로 조용히 대체할지도 명시해야 한다.

스터디에서 확인할 질문

project config가 malformed일 때 전체 시작을 막을지 해당 key만 무시할지 어떤 기준으로 결정할 것인가?

migration은 오래된 durable state와 새 코드의 handshake다

모델 alias와 설정 key가 바뀌어도 사용자의 이전 선택을 의미 있게 옮겨야 한다.

migrations 디렉터리에는 Fennec alias를 Opus로 옮기거나 기본값을 재설정하는 작은 변환이 있다. 각각은 과거 제품 의미를 현재 vocabulary로 번역한다.

migration을 파일 이름과 실행 횟수로만 보면 위험하다. idempotency, partial failure, version marker, rollback이 필요하다. 동일 process에서 여러 번 실행돼도 결과가 안정적이어야 한다.

migration code는 시간이 지나도 historical contract로 남는다. 현재 이름이 낯설다는 이유로 삭제하면 오래된 설치의 upgrade path가 끊긴다. removal은 지원 최소 버전 evidence와 함께 해야 한다.

스터디에서 확인할 질문
fennec-fast-latest처럼 하나의 alias가 model과 fast mode 두 설정으로 분해될 때 atomicity를 어떻게 보장할 것인가?

command catalog는 운영 기능의 발견 표면이다

80개가 넘는 slash command는 제품 설정과 진단, onboarding, extension 관리를 노출한다.

commands tree는 UI 기능 목록이면서 내부 application service registry다. feature gate와 user type에 따라 숨겨진 command도 있다.

많은 command가 있다는 사실은 중요도를 뜻하지 않지만 운영 복잡도를 보여 준다. 같은 설정을 command, UI panel, config file이 각각 변경하면 write path가 여러 개가 된다.

구조 개선은 command를 폴더별로 정리하는 데서 끝나지 않는다. underlying use case를 하나로 두고 command와 UI를 adapter로 만들면 validation과 telemetry가 일치한다.

스터디에서 확인할 질문

설정 변경 command 하나를 골라 동일 값을 바꾸는 다른 UI/API 경로가 있는지 찾아보라.

03

Telemetry와 비용 코드는 운영의 두 번째 모델이다

실행 코드가 무엇을 하는지와 운영 코드가 무엇을 관찰하는지는 다르며, 관찰 schema가 팀의 문제 인식을 형성한다.

이 장에서 실제로 펼쳐볼 경로

src/services/analytics/** · src/cost-tracker.ts · src/services/api/**

analytics pipeline은 event 생산과 sink를 분리한다

동일 event가 first-party exporter와 제한된 Datadog sink로 다른 policy를 따라간다.

analytics 디렉터리에는 metadata 수집, event logger, exporter, sink killswitch가 나뉜다. batch, retry, disk persistence는 네트워크 장애에서도 전달을 지속하게 한다.

이 구조는 observability reliability를 높이지만 privacy와 retention surface도 넓힌다. 운영 가치와 데이터 최소화는 같은 방향으로 자동 정렬되지 않는다.

분석할 때 endpoint 목록보다 event taxonomy, common metadata, truncation, opt-out condition을 보자. 어떤 사건이 항상 session/user identifier와 결합되는지가 더 중요한 질문일 수 있다.

운영 event가 전송될 때까지

01	→	02	→	03	→	04	→	05
Produce		**Enrich**		**Buffer**		**Persist/Retry**		**Sink**
domain event와 correlation id		공통 environment metadata		batch·size·time window		실패 event를 disk에 보존		1P/Datadog별 policy

이 그림이 감추지 않는 것 reliable delivery는 data minimization과 별도 품질이다. retry와 disk persistence가 강화될수록 retention과 access review가 더 중요해진다.

스터디에서 확인할 질문

오류 진단에 필요하지만 사용자 식별과 결합할 필요가 없는 metadata를 어떻게 분리할 수 있는가?

metadata.ts는 환경을 운영 언어로 번역한다

platform, runtime, subscription, repo hash가 모든 event의 분석 차원을 만든다.

저장소 자체 문서는 넓은 metadata 수집과 tool input truncation을 지적한다. 코드를 보면 어떤 field가 기본, opt-in, hashed, truncated인지 구체적으로 확인할 수 있다.

해시는 익명화와 동일하지 않다. 안정된 repo hash는 여러 session을 연결할 수 있고 작은 후보 집합에서는 재식별 가능성이 있다. 반대로 진단에는 environment correlation이 유용하다.

privacy review는 "원문을 보내는가"만 묻지 말고 linkability, retention, access, purpose를 함께 본다. tool detail 환경 변수처럼 운영자가 켤 수 있는 상세 로그도 별도 threat가 된다.

스터디에서 확인할 질문

동일 버그를 재현하는 데 필요한 최소 environment fingerprint는 무엇이며 어떤 field는 제거할 수 있는가?

비용 추적은 product policy와 protocol state를 잇는다

token usage와 모델 가격을 합산해 budget stop, UI 표시, SDK result에 사용한다.

Cost tracker는 모델별 가격과 usage를 금액으로 바꾼다. prompt cache read/write, thinking token, fallback model이 있으면 단순 input/output 곱셈보다 복잡하다.

비용은 추정치일 수 있으며 provider billing과 완전히 같다고 가정하면 안 된다. 그렇지만 turn 중 budget enforcement에는 빠른 local estimate가 필요하다.

모델 출시 때 가격표와 context window를 갱신하라는 marker가 있는 이유다. 운영 상수 변경은 기능 코드 변경만큼 test와 review가 필요하다.

스터디에서 확인할 질문
가격 정보가 오래됐을 때 hard budget stop을 계속 적용하는 것이 사용자에게 더 안전한가?

좋은 계측은 effect와 outcome 사이의 join을 남긴다

tool_use count만으로 코드 변경 성공이나 사용자 가치를 알 수 없다.

이 코드에는 많은 event가 있지만 activity 수가 correctness를 증명하지 않는다. permission denial, tool result, session outcome을 같은 correlation id로 연결해야 실패 원인을 설명할 수 있다.

반대로 모든 것을 join하기 위해 raw prompt와 path를 수집하면 privacy를 해친다. 최소한의 stable id와 aggregate outcome으로 질문에 답할 수 있는지 먼저 검토해야 한다.

계측을 추가할 때 "어떤 결정을 위해 필요한가"를 적으면 불필요한 field와 무기한 retention을 줄일 수 있다. dashboard 존재가 measurement validity를 보장하지 않는다.

스터디에서 확인할 질문

Edit 실패율을 개선하려면 event에 어떤 phase와 reason이 필요하며 source code 내용은 왜 필요하지 않은가?

04

연구 문서에서 구조 개선 과제로 돌아온다

발견 목록을 그대로 믿거나 버리지 않고, 그 문서의 탐색법을 core code와 연결해 검증 가능한 리팩터링 질문으로 바꾼다.

이 장에서 실제로 펼쳐볼 경로

docs/ko/** · src/query.ts · src/screens/REPL.tsx · src/services/api/claude.ts

저장소의 기존 보고서는 symbol-led investigation을 사용한다

특이한 flag·endpoint·codename에서 시작해 정의와 call site를 모으고 주제를 만든다.

텔레메트리 문서는 exporter와 metadata, 원격 제어 문서는 polling과 killswitch, roadmap 문서는 gate와 prompt marker를 묶는다. 이 방식은 넓은 복원 코드에서 숨은 subsystem을 발견하는 데 효과적이다.

한계는 discovery가 곧 importance로 보일 수 있다는 점이다. 눈에 띄는 코드네임보다 QueryEngine의 상태 보존이 제품 핵심일 수 있다. 또한 comment는 계획 증거가 아니라 해당 snapshot의 text evidence다.

새 6권은 기존 문서의 발견을 운영/구성 권에 흡수하되 전체 분류는 runtime responsibility로 다시 세웠다. 발견은 장을 지배하지 않고 해당 core를 설명할 때 사용된다.

스터디에서 확인할 질문

기존 5개 보고서에서 관찰 사실과 미래 추론이 한 문단에 섞인 사례를 하나 찾아 다시 작성해 보라.

큰 파일은 책임 분포를 찾는 지도이지 자동 분해 명령이 아니다

query.ts, main.tsx, REPL.tsx, claude.ts는 서로 다른 변화가 한 지점에 모였다는 신호다.

각 큰 파일에서 import cluster, state owner, error handling, consumer를 보면 후보 seam이 나온다. query는 turn protocol, main은 composition, REPL은 projection controller, claude.ts는 provider adapter로 중심 성격도 다르다.

모두 같은 "service extraction" 처방을 쓰면 의미가 나빠질 수 있다. state machine은 transition owner를, composition은 dependency graph를, adapter는 normalization contract를 중심으로 나눠야 한다.

분해 전에는 characterization trace를 만든다. happy path output뿐 아니라 cancellation, permission denial, resume, cache behavior를 포함해야 한다.

같은 "큰 파일"에 다른 분해 기준이 필요한 이유

파일	중심 성격	좋은 seam
main.tsx	composition	초기화 산출물과 mode
query.ts	state machine	transition/settlement owner
REPL.tsx	projection controller	read model과 command
claude.ts	provider adapter	normalization/retry boundary

구분해야 하는 이유 line count는 탐색 신호일 뿐이다. 변경 이유와 invariant가 다른데 같은 extraction 처방을 쓰면 coupling을 이동시키기만 한다.

스터디에서 확인할 질문

네 큰 파일 중 하나를 골라 변경 이유 세 가지와 그 사이 공유 invariant를 적어 보라.

리팩터링은 이동 순서가 설계의 일부다

공통 contract와 관찰을 먼저 만들지 않으면 파일을 옮기는 동안 오류를 발견할 기준이 사라진다.

안전한 순서는 대체로 현재 behavior trace 확보, typed seam 도입, 한 책임 이동, dual/shadow 비교, old path 제거다. 실제 effect는 두 번 실행하지 않고 record/replay adapter로 비교한다.

대규모 cleanup과 기능 변경을 함께 하면 실패 원인을 분리하기 어렵다. 특히 복원 source는 upstream tests와 history가 부족하므로 작은 변경과 명시적 limitation이 더 중요하다.

구조 개선 목표는 파일 수가 아니라 change consequence다. provider 추가가 query core를 덜 건드리고, 새 UI가 session format을 바꾸지 않게 되는지가 결과다.

스터디에서 확인할 질문

Tool pipeline을 분리할 때 가장 먼저 고정해야 할 characterization test는 무엇인가?

이 프로젝트를 이해했다는 기준은 변경을 예측하는 능력이다

파일 위치를 기억하는 것보다 새 요구가 어느 owner와 invariant를 건드리는지 설명할 수 있어야 한다.

예를 들어 “원격 SDK에 새로운 progress event 추가” 요구는 QueryEngine projection, control schema, bridge backpressure, REPL compatibility를 건드린다. “Edit에 hash version 추가”는 Read cache, validation, transcript, error UX를 건드린다.

이 예측은 상위 흐름 나열에서 나오지 않는다. 타입 메커니즘, 수명, identity, authority, projection을 함께 이해할 때 가능하다.

6권의 마지막에는 각 팀원이 change scenario를 하나 제시하고 영향 graph를 그리는 방식이 좋다. 다른 팀원이 빠진 invariant와 evidence를 질문하면 학습이 코드 리뷰 능력으로 이어진다.

스터디에서 확인할 질문

새 provider를 추가한다는 요구에 대해 반드시 유지해야 할 내부 contract 다섯 개를 말할 수 있는가?

코드 뒤에서 작동하는 언어·런타임 메커니즘

문법 이름을 외우기보다, 런타임이 실제로 무엇을 저장하고 언제 제어를 넘기는지와 이 프로젝트가 그것을 어떤 책임에 사용하는지를 함께 본다.

compile-time intrinsic

언어가 제공하는 것
bundler가 표현을 상수로 치환하고 도달 불가능 branch를 제거한다.

이 저장소가 쓰는 방식
feature()와 MACRO가 배포 variant를 만든다.

놓치기 쉬운 함정
runtime flag처럼 생각하면 누락 모듈을 나중에 켤 수 있다고 오해한다.

dead-code elimination

언어가 제공하는 것
정적으로 false인 branch와 참조를 bundle에서 제거한다.

이 저장소가 쓰는 방식
내부 전용 기능을 외부 npm artifact에서 제외한다.

놓치기 쉬운 함정
복원 source의 dangling import만으로 제거된 구현을 재구성할 수 없다.

ESM module graph

언어가 제공하는 것
static import가 bundle resolution과 side-effect order를 결정한다.

이 저장소가 쓰는 방식
entrypoint와 provider/service dependency를 구성한다.

놓치기 쉬운 함정
false branch 안 import도 도구에 따라 resolution될 수 있어 stub이 필요해진다.

configuration precedence

언어가 제공하는 것
여러 source의 value를 정해진 우선순위와 validation으로 합친다.

이 저장소가 쓰는 방식
default/user/project/managed 설정이 effective behavior를 만든다.

놓치기 쉬운 함정
object spread만 남기면 값의 source와 override 불가 이유가 사라진다.

idempotent migration

언어가 제공하는 것
같은 변환을 반복해도 한 번 적용한 것과 같은 durable state를 만든다.

이 저장소가 쓰는 방식
model alias와 설정 key의 역사적 의미를 새 버전으로 옮긴다.

놓치기 쉬운 함정
부분 write와 복합 설정 분해가 원자적이지 않으면 중간 상태가 남는다.

batched telemetry exporter

언어가 제공하는 것
event를 buffer·batch·retry하고 실패 데이터를 disk에 보존한다.

이 저장소가 쓰는 방식
운영 분석을 network 장애 뒤에도 전달한다.

놓치기 쉬운 함정
신뢰성이 높아질수록 privacy·retention surface도 넓어진다.

스터디용 코드 탐색 인덱스

파일을 처음부터 끝까지 읽지 않는다. 아래 시작점에서 symbol을 잡고 정의·호출·상태·실패 경로를 왕복한다.

시작 경로	먼저 볼 표현	그 다음에 물을 질문
README_KR.md	출처/누락/통계	repository 주장과 직접 측정이 일치하는가
scripts/prepare-src.mjs	in-place transform	연구 source provenance를 바꾸는가
scripts/build.mjs	copy/transform/stub	build 성공과 semantic equivalence를 구분하는가
stubs/bun-bundle.ts	feature fallback	원래 build-time value의 evidence인가 단지 도구인가
src/utils/settings	merge/source metadata	effective value의 provenance가 남는가
src/migrations	version transform	idempotency와 partial failure를 다루는가
src/services/analytics/metadata.ts	공통 metadata	진단 목적에 비해 collection이 최소인가
docs/ko	symbol-led claims	관찰과 production/roadmap 추론이 분리되는가

> **코드 독해 규칙**
>
> 타입 선언만 보고 runtime 검증을 가정하지 않는다. 호출 순서만 보고 authority를 가정하지 않는다. UI 표시만 보고 durable state를 가정하지 않는다. 주석만 보고 production 활성이나 upstream 의도를 확정하지 않는다.

복원 코드에서 검증 가능한 리팩터링 제안 만들기

기존 문서의 발견 하나를 core responsibility와 연결하고, 코드 밖 추론을 분리한 뒤 작은 구조 개선으로 바꾼다.

1. 기존 `docs/ko/**` 보고서에서 주장 하나를 고르고 관찰·추론·unknown으로 문장을 나눈다.
2. 관련 symbol의 definition, call site, configuration source, telemetry를 고정 commit에서 추적한다.
3. 그 기능이 여섯 responsibility 중 어느 owner에 속하고 어떤 cross-cutting concern과 만나는지 적는다.
4. change scenario와 failure scenario를 하나씩 만들고, 현재 구조에서 수정해야 할 파일을 예측한다.
5. typed seam, validation oracle, 단계적 이동 순서를 포함한 리팩터링 제안을 작성한다.

> **완료의 기준**
>
> “큰 파일을 나누자”가 아니라 특정 변경의 파급을 줄이면서 기존 protocol과 authority를 보존하는 제안을 낼 수 있으면 된다.

다음 모임에 남길 기록

스터디 전체의 마지막 기록은 가장 확실한 사실, 가장 유용한 해석, 아직 확인할 수 없는 질문을 각각 하나씩 남긴다.